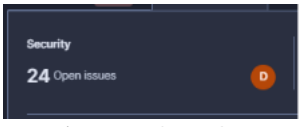
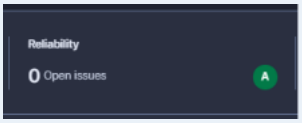
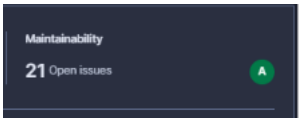
Reporte DAST

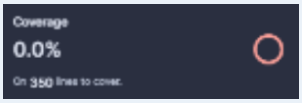
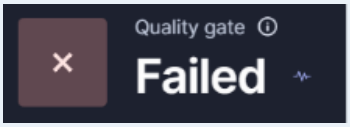
Fases 1 y 2 · un reporte por grupo

0. Mapeo de enlaces y archivos

	Descripción	Archivo: línea o URL
Zap	Reportes de ZAP: línea base (Fase 1) y escaneos baseline y full scan de la Fase 2	Fase 1/reporte-zap.html · Fase 2/evidencias/reporte-zap-baseline-final.html · Fase 2/evidencias/reporte-zap-full-final.html
Sonarqube	Reporte únicamente con sonarqube	Fase 1 solo sonarqube.pdf
	Reporte sonarqube y samgrep	Fase 1 sonarqube y samgrep.pdf
	Reporte con sonarqube, samgrep y PUNTOS EXTRAS de Owasp dependecy check	Fase 1 sonarqube, samgrep y puntos extras owasp dependecy check.pdf
	Capturas del análisis final de SonarQube (Fase 2)	Fase 2/evidencias/capturas/
Repositorio	Repositorio del proyecto con los commits de las correcciones. Es una copia con el historial completo de andreetoledo/secure-crm-dastsast, que no daba permisos de escritura.	https://github.com/Maria-Villafuerte/secure-crm-dastsast
Evidencias	Salidas de pruebas, auditoría, Semgrep, cabeceras y capturas de la Fase 2	Fase 2/evidencias/ y Fase 2/semgrep-fase2.sarif

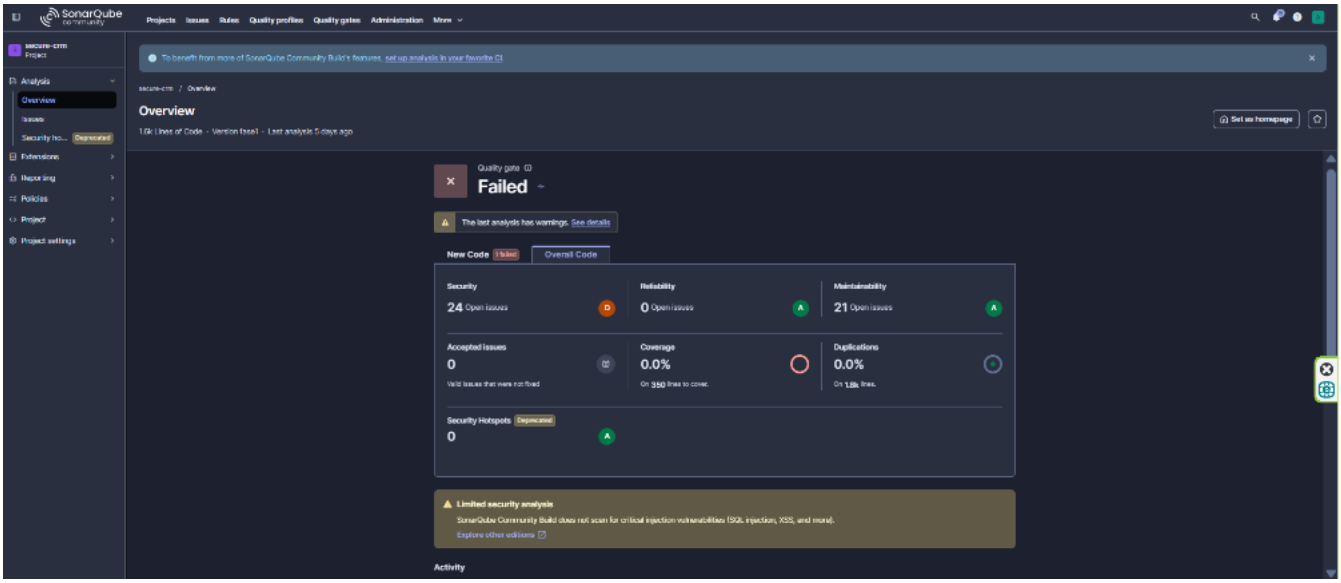
1. Línea base (Fase 1)

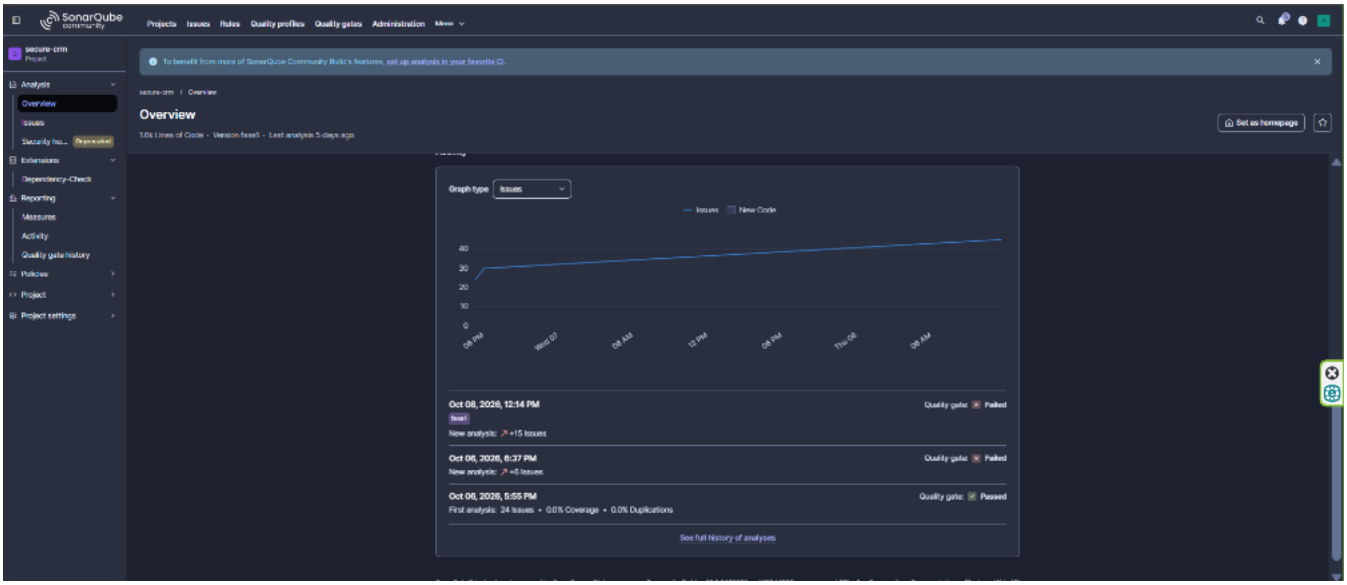
Métrica	Fase 1	Fase 2
Security (rating / issues)	 D / 24 issues (3 SonarQube + 6 Semgrep + 15 Dependency-Check)	A / 0 issues
Reliability (rating / issues)	 A/0	C / 6 issues (5 bugs). Reglas nuevas de SonarQube 26.10 (S9383, S9379) sobre páginas del frontend que no cambiaron; también aparecen en fase1 reanalizado, por eso no son código nuevo.
Maintainability (rating / issues)	 A/ 21 issues	A / 1 issue

Cobertura total	 0.0% 350 lines to cover	58.9 % (829 líneas por cubrir): la API tiene 71 % y el frontend 26 %, porque las páginas de React no tienen pruebas.
Cobertura del código nuevo	—	90.5 % (166 líneas nuevas)
Quality Gate	Failed 	Passed (análisis fase2, commit 713aaf2)
Dependencias vulnerables (auditoría)	pnpm audit --prod: 50 vulnerabilidades (5 críticas, 17 altas, 24 moderadas, 4 bajas). pnpm audit completo: 53 (6 críticas, 18 altas, 25 moderadas, 4 bajas). Dependency-Check: 15 (10 de paquetes npm de la app y 5 de librerías Java del plugin de SonarQube).	pnpm audit --prod: 0 vulnerabilidades (antes 50). pnpm audit completo: 3, todas de herramientas de desarrollo y aceptadas: shell-quote (#18), braces (#19) y postcss-selector-parser (#22). Versiones en pnpm-lock.yaml: proxy-addr 2.0.8, next 15.5.28, jsonwebtoken 9.0.3, lodash 4.18.1, qs 6.16.1, postcss 8.5.26 y source-map-js ≥ 1.2.2. Los Dockerfile instalan con --frozen-lockfile, así que la imagen usa exactamente esas versiones.
Alertas altas/medias de ZAP	0 altas / 2 medias (CSP ausente; anti-clickjacking ausente)	Baseline: 0 altas / 0 medias / 1 baja (COEP, aceptada #11). Full scan: 0 altas / 0 medias / 1 baja, 141 reglas pasadas. Detalle alerta por alerta en la sección 6.2.

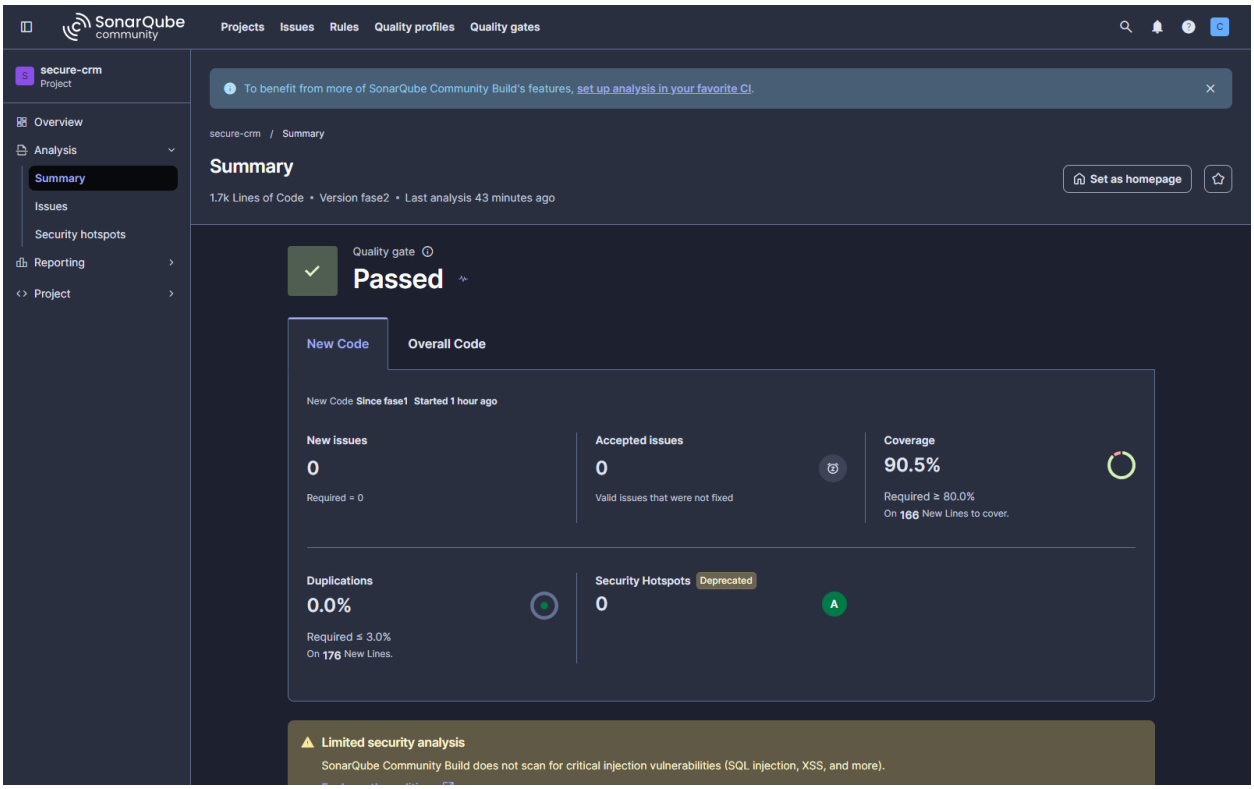
Nota sobre el reanálisis. El SonarQube de los análisis originales estaba en otra computadora. El 16 de octubre se reconstruyó el historial en una instancia nueva, con el gate «Curso Desarrollo Seguro» y la misma secuencia: fase1 sobre el código original (commit 848d4be), fase1 con Semgrep importado y fase2 sobre el código corregido (commit 713aaf2, analizado con git). Las capturas finales son de ese reanálisis.

Captura del dashboard de la Fase 1 (análisis inicial del proyecto):



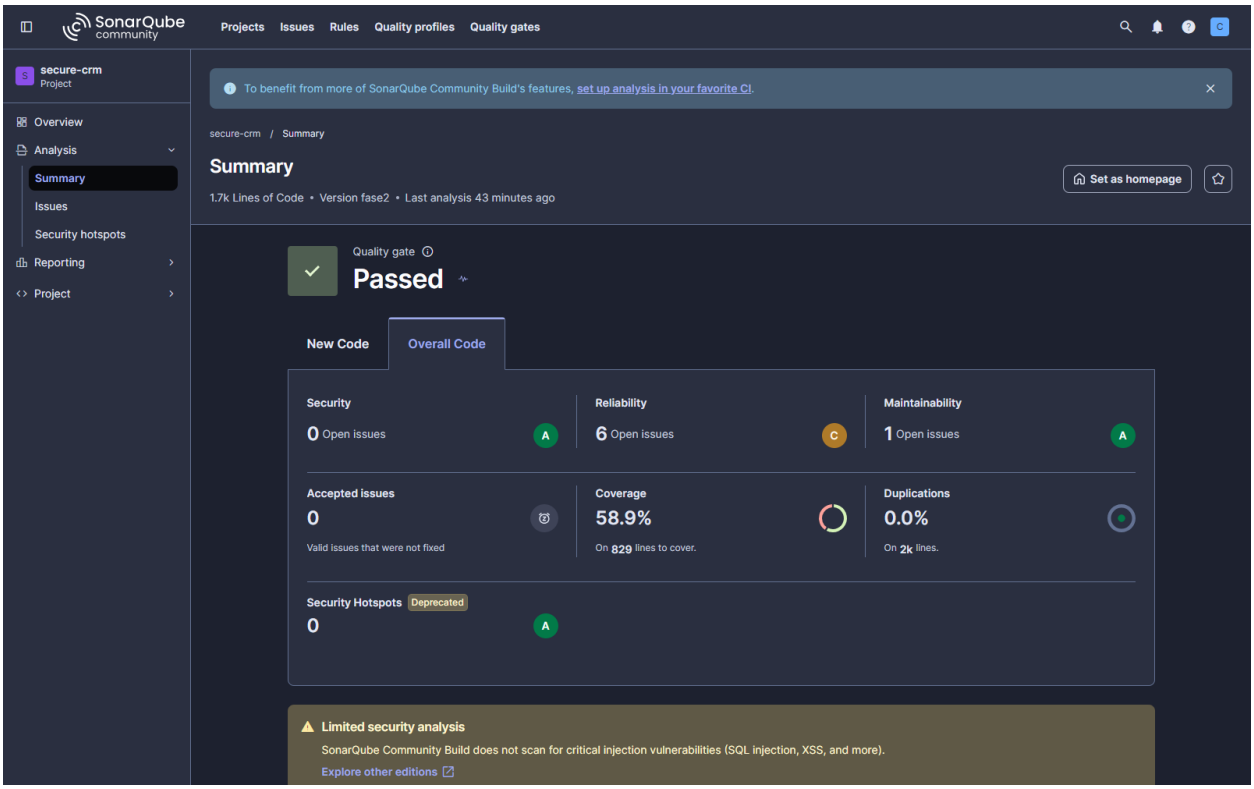


Captura del dashboard del reporte final (análisis fase2, commit 713aaf2), pestaña New Code:



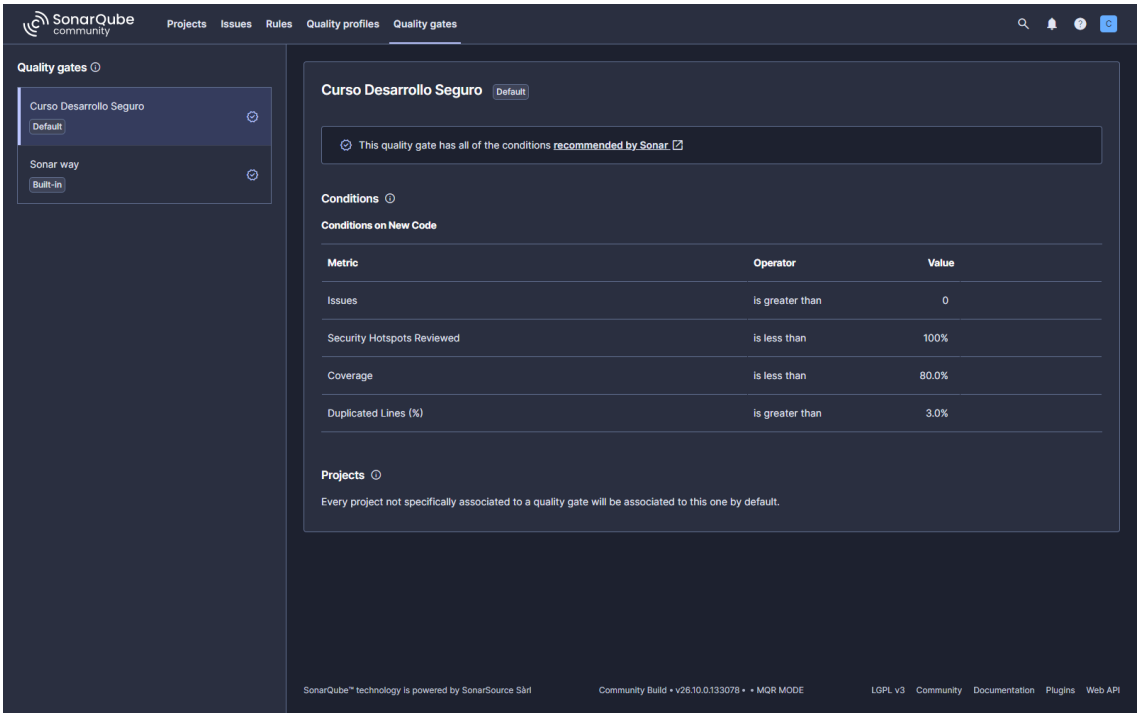
Quality Gate Passed: 0 issues nuevos, 90.5 % de cobertura y 0 % de duplicación en el código nuevo.

Pestaña Overall Code:

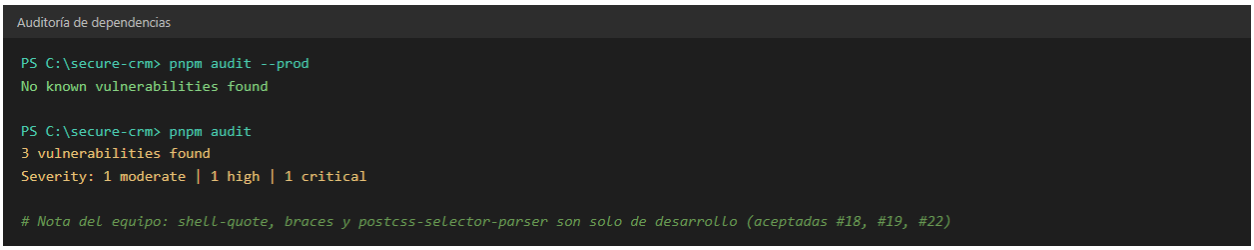


Security A con 0 vulnerabilidades; Reliability C por 6 issues que no son código nuevo.

Quality Gate «Curso Desarrollo Seguro» (predeterminado):



Captura de pnpm audit --prod (0 vulnerabilidades en producción):



2. Hallazgos de seguridad

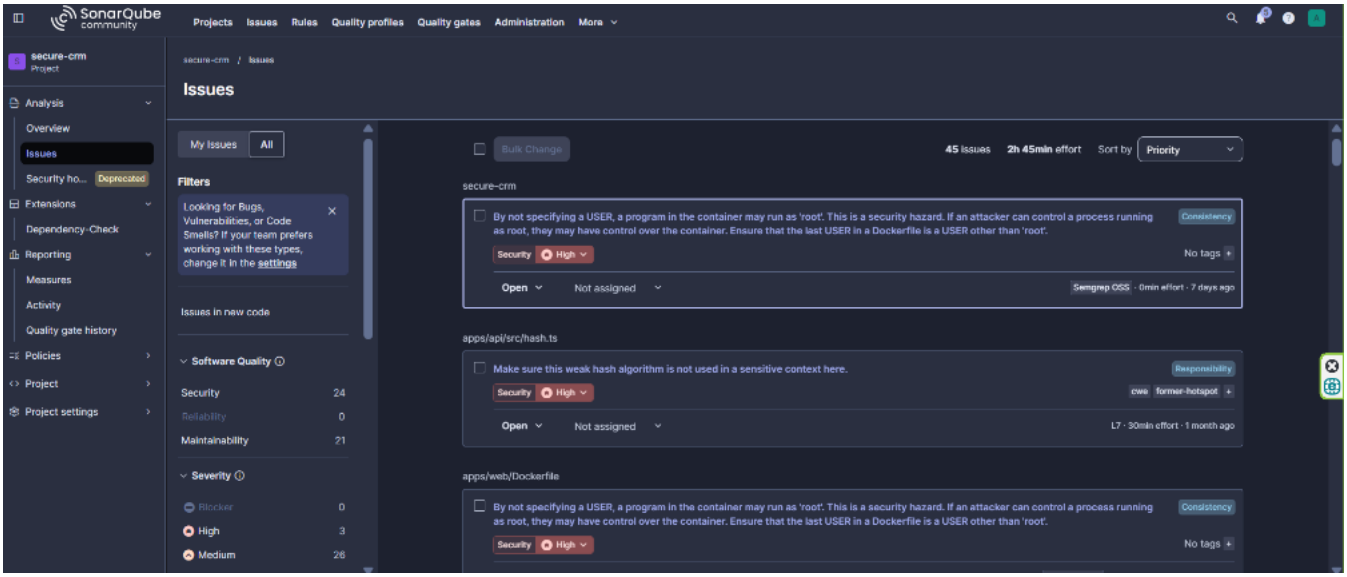
Una fila por hallazgo. **Fuente:** SonarQube, Semgrep, Auditoría, Dependency-Check o ZAP.

#	Hallazgo y fuente	Archivo:línea o URL	CWE / OWASP 2025	CVSS v3.1 (vector y puntaje)	Decisión
1	MD5 usado para hashear contraseñas (SonarQube `typescript:S4790` · Semgrep `md5-used-as-password`)	apps/api/src/hash.ts:7	CWE-327, CWE-916 / A04	AV:N - AC:H - PR:N - UI:N - S:U - C:H - I:N - A:N = 5.9	Corregir
2	Contenedor web corre como root (SonarQube `docker:S6471` · Semgrep `missing-user`)	apps/web/Dockerfile:3 y :26	CWE-250 / A02	AV:L - AC:H - PR:L - UI:N - S:C - C:L - I:L - A:L = 5.3	Corregir
3	Cabecera `X-Powered-By` expone el framework (SonarQube `typescript:S5689` · ZAP)	apps/api/src/index.ts:12 http://host.docker.internal:3000/ (cabecera de respuesta; sin parámetro)	CWE-497 / A02	AV:N - AC:L - PR:N - UI:N - S:U - C:L - I:N - A:N = 5.3	Corregir
4	pnpm sin `trustPolicy: no-downgrade` (Semgrep `pnpm-trust-policy`)	pnpm-workspace.yaml	CWE-1357 / A03	N/A (endurecimiento; no es explotable por sí sola)	Corregir
5	pnpm sin `blockExoticSubdeps: true` (Semgrep)	pnpm-workspace.yaml	CWE-829 / A03	N/A (endurecimiento; no es explotable por sí sola)	Corregir
6	pnpm sin `minimumReleaseAge` (Semgrep)	pnpm-workspace.yaml	CWE-1357 / A03	N/A (endurecimiento; no es explotable por sí sola)	Corregir
7	Content Security Policy (CSP) Header Not Set (ZAP, Medium)	http://host.docker.internal:3000/ y /robots.txt, /sitemap.xml (4 instancias; sin parámetro)	CWE-693 / A02	AV:N - AC:H - PR:N - UI:R - S:U - C:L - I:L - A:N = 4.2	Corregir
8	Missing Anti-clickjacking Header (ZAP, Medium)	http://host.docker.internal:3000/ (2 instancias; parámetro x-frame-options)	CWE-1021 / A02	AV:N - AC:L - PR:N - UI:R - S:U - C:N - I:L - A:N = 4.3	Corregir
9	`X-Content-Type-Options` ausente (ZAP, Low)	http://host.docker.internal:3000/ y recursos /_next/static/... (parámetro x-content-type-options).	CWE-693 / A02	AV:N - AC:H - PR:N - UI:R - S:U - C:L - I:N - A:N = 3.1	Corregir
10	`Cross-Origin-Opener-Policy` y `Cross-Origin-Resource-Policy` ausentes (ZAP, Low)	http://host.docker.internal:3000/ y /_next/static/... (parámetros Cross-Origin-Opener-Policy y Cross-Origin-Resource-Policy).	CWE-693 / A02	AV:N - AC:H - PR:N - UI:R - S:U - C:L - I:N - A:N = 3.1	Corregir
11	`Cross-Origin-Embedder-Policy` ausente (ZAP, Low)	http://host.docker.internal:3000/ (parámetro Cross-Origin-Embedder-Policy).	CWE-693 / A02	AV:N - AC:H - PR:N - UI:R - S:U - C:L - I:N - A:N = 3.1	Aceptar
12	Permissions Policy Header Not Set (ZAP, Low)	#12 Permissions Policy: http://host.docker.internal:3000/ y recursos /_next/static/..., /robots.txt, /sitemap.xml.	CWE-693 / A02	AV:N - AC:H - PR:N - UI:R - S:U - C:L - I:N - A:N = 3.1	Corregir

13	`proxy-addr` · GHSA-jqcg-44mw-7w3h (Dependency-Check)	pnpm-lock.yaml (paquete proxy-addr)	CWE-290, CWE-348, CWE-697 / A03	AV:N - AC:L - PR:N - UI:N - S:U - C:H - I:H - A:N = 9.1	Corregir
14	`next` · GHSA-g5qg-72qw-gw5v (Dependency-Check)	apps/web (dependencia directa)	CWE-524 / A03	AV:L - AC:L - PR:N - UI:N - S:U - C:H - I:N - A:N = 6.2	Corregir
15	`jsonwebtoken` · GHSA-hjrf-2m68-5959 (Dependency-Check)	apps/api (dependencia directa)	CWE-287, CWE-1259 / A03 (también A07)	AV:N/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:L · 5.0	Corregir
16	`lodash` · GHSA-r5fr-rjxr-66jc (Dependency-Check)	pnpm-lock.yaml (paquete lodash)	CWE-94 / A03	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H · 8.1	Corregir
17	`qs` · GHSA-4mjr-xmp4-gh2g (Dependency-Check)	pnpm-lock.yaml (paquete qs)	CWE-248, CWE-703 / A03	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L · 5.3	Corregir
18	`shell-quote` · GHSA-pqg4-j6r4-53mv (Dependency-Check)	pnpm-lock.yaml (paquete shell-quote)	CWE-78 / A03	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H · 8.1	Aceptar
19	`braces` · GHSA-vfj7-8cjw-p6xm (Dependency-Check)	pnpm-lock.yaml (paquete braces)	CWE-674 / A03	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H · 7.5	Aceptar
20	`postcss` · GHSA-6g55-p6wh-862q (Dependency-Check)	pnpm-lock.yaml (paquete postcss)	CWE-22, CWE-200 / A03	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N · 7.5	Corregir
21	`source-map-js` · GHSA-68fv-2mgg-jv7q (Dependency-Check)	pnpm-lock.yaml (paquete source-map-js)	CWE-1284 / A03	AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H · 7.5	Corregir
22	`postcss-selector-parser` · GHSA-rj75-hqrm-r3gf (Dependency-Check)	pnpm-lock.yaml (paquete postcss-selector-parser)	CWE-400, CWE-407 / A03	AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:N/A:H · 5.9	Aceptar

Decisión: Corregir · Aceptar · Falso positivo.

Captura de hallazgos:



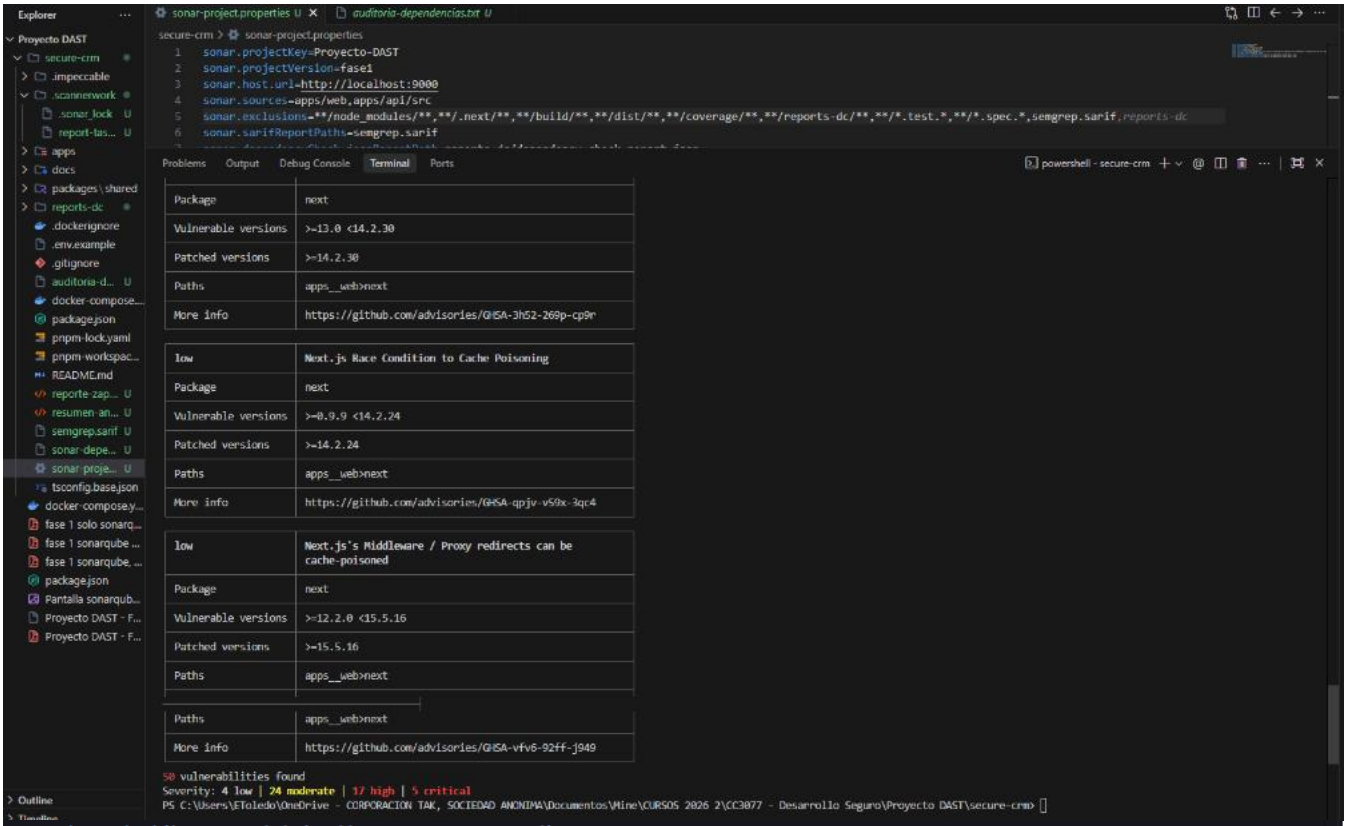
Justificación de los aceptados y falsos positivos

#	Justificación
11	**Aceptado.** COEP (`require-corp`) obliga a que todo recurso de otro origen (fuentes, imágenes, CDN) envíe CORP/CORS; activarlo puede romper la aplicación sin una ganancia real, porque solo hace falta para aislamiento de origen cruzado (`SharedArrayBuffer`, temporizadores de alta precisión) y el CRM no los usa. El riesgo residual es bajo (CVSS 3.1) y se mitiga con COOP y CORP (#10). Se reevalúa si la aplicación incorpora esas APIs.
18	**Aceptado (temporal).** La falla de `shell-quote` (CWE-78) solo es explotable si se invoca con entrada controlada por un atacante. La aplicación no construye comandos de shell en ejecución, y el paquete llega como dependencia de herramientas de desarrollo. Condición verificable: no debe aparecer en `pnpm audit --prod`; si aparece, pasa a Corregir (≥ 1.11.0). Se actualiza en el próximo ciclo de dependencias.
19	**Aceptado (temporal).** `braces` es de compilación/globbing y su falla (recursión no controlada, DoS) requiere patrones hostiles que un atacante no puede enviar a la aplicación en ejecución. El advisory no publica versión corregida, así que no hay actualización posible hoy. Condición: no aparece en `pnpm audit --prod`. Se revisa cuando exista parche.
22	**Aceptado (temporal).** El ReDoS de `postcss-selector-parser` (CWE-400) se activa al parsear selectores CSS complejos en el build; ningún dato de usuario llega a ese parser en ejecución. Condición: no aparece en `pnpm audit --prod`. Se actualiza (≥ 7.1.6) con el resto de la toolchain CSS.

3. Dependencias vulnerables

Paquete	Severidad	Camino (quién la trae)	¿Alcanzable en ejecución?	Decisión
proxy-addr	Crítica (9.1)	apps/api > express > proxy-addr	Condicional: afecta si `trust proxy` usa subredes IPv4 y la lógica usa `req.ip` (límites por IP, logs, controles)	Corregido: proxy-addr 2.0.8 (override ^2.0.8) y trust proxy 1
next	Moderada (6.2)	apps/web (directa)	Sí: es el servidor del frontend, atiende cada petición; la falla de caché aplica si hay contenido sensible cacheado	Corregido: next 15.5.28 (^15.5.24) con React 19
jsonwebtoken	Moderada (5.0)	apps/api (directa)	Sí, si la API verifica JWT en cada petición autenticada (afecta la autenticación)	Corregido: jsonwebtoken 9.0.3 con algorithms: ['HS256']
lodash	Alta (8.1)	`apps/api > lodash (directa)`	Solo si se usa la ruta de plantillas (`_.template`) con datos del usuario	Corregido: lodash 4.18.1 (override ^4.18.0)
qs	Moderada (5.3)	apps/api > express > qs	Parcial: el parseo de peticiones no usa `stringify`; solo si el código lo llama con datos del usuario	Corregido: qs 6.16.1 (override ^6.16.0)
shell-quote	Alta (8.1)	`raíz > concurrently > shell-quote`	No está en producción: la app no arma comandos de shell	Aceptar (temporal)
braces	Alta (7.5)	`apps/web > tailwindcss > chokidar > braces`	No en producción: requiere patrones hostiles; solo compilación	Aceptar (sin parche publicado)
postcss	Alta (7.5)	`apps/web > next > postcss`	Sí	Corregido: postcss 8.5.26 vía override (next fija 8.4.31)
source-map-js	Alta (7.5)	apps/web > next > postcss > source-map-js	Sí	Corregido: source-map-js ≥ 1.2.2 vía override
postcss-selector-parser	Moderada (5.9)	`apps/web > tailwindcss > postcss-selector-parser`	No en producción: ReDoS solo al compilar CSS	Aceptar; actualizar con la toolchain CSS

Captura de hallazgos:




```
PS C:\Users\Fabi\Documents\U\GitHub\secure-crm> pnpm audit
```

Package	next
Vulnerable versions	>=12.2.0 <15.5.16
Patched versions	>=15.5.16
Paths	apps__web>next
More info	https://github.com/advisories/GHSA-3g8h-86w9-wvmq

low	Next.js vulnerable to cache poisoning via collisions in React Server Component cache-busting
Package	next
Vulnerable versions	>=13.4.6 <15.5.16
Patched versions	>=15.5.16
Paths	apps__web>next
More info	https://github.com/advisories/GHSA-vfv6-92ff-j949

```
53 vulnerabilities found
Severity: 4 low | 25 moderate | 18 high | 6 critical
```

4. SAST vs. DAST

Para cada vulnerabilidad que encontró ZAP:

Alerta de ZAP (URL, parámetro)	¿La vio SAST? ¿Qué herramienta?	Línea de código que la causa
CSP no configurada (Medium) http://localhost:3000/ · parámetro: ninguno (cabecera)	**No.** Ninguna regla de SonarQube ni de Semgrep detecta la falta de una cabecera.	Ausencia de código: `headers()` sin CSP en `next.config.js` (web) y sin `helmet()` en `apps/api/src/index.ts` (API).
Falta anti-clickjacking (Medium) http://localhost:3000/ · parámetro: ninguno	**No.**	Ausencia de `X-Frame-Options` / `frame-ancestors`: mismos archivos que la CSP.
X-Content-Type-Options ausente (Low) http://localhost:3000/ · ninguno	**No.**	Ausencia de `nosniff` en `next.config.js` / `index.ts` (`helmet` lo agrega por defecto).
Server leaks «X-Powered-By» (Low) http://localhost:3000/ · ninguno	**Parcial.** SonarQube (`typescript:S5689`) lo vio en la API Express; el de :3000 corresponde al frontend, que ningún SAST marcó.	`apps/api/src/index.ts:12` (`express()` sin `app.disable("x-powered-by")`) y `poweredByHeader` por defecto en `next.config.js`.
COOP / COEP / CORP ausentes (Low) http://localhost:3000/ · ninguno	**No.**	Ausencia de las tres cabeceras (COOP y CORP las agrega `helmet`; COEP se acepta, ver #11).
Permissions Policy ausente (Low) http://localhost:3000/ · ninguno	**No.**	Ausencia de `Permissions-Policy` en `next.config.js` / `index.ts`.

Reflexión: ¿qué encontró cada herramienta que las otras no?

- SAST (SonarQube + Semgrep) encontró lo que está escrito en el repositorio: MD5 para contraseñas (hash.ts:7), el contenedor sin USER y los ajustes faltantes de pnpm. Ubica archivo y línea, pero no ve las cabeceras que realmente salen del servidor.
- SCA (Dependency-Check y la auditoría de pnpm) vio las CVEs de 10 paquetes (hasta CVSS 9.1), algo que ni SAST ni DAST ven, pero no sabe si el código vulnerable se ejecuta; por eso se priorizó por alcanzabilidad.
- DAST (ZAP) encontró lo que solo existe al ejecutar: la falta de CSP, anti-clickjacking, nosniff, COOP/COEP/CORP y Permissions-Policy, porque dependen de la respuesta HTTP final (Next.js + Express), no de una línea concreta. No ve MD5 ni las dependencias. Solo X-Powered-By tuvo coincidencia parcial entre SAST y DAST.
- Ninguna herramienta reportó fallas de control de acceso o de lógica de negocio; eso no demuestra que no existan, sino que requieren pruebas manuales o escaneos autenticados. Las tres técnicas son complementarias.

5. Propuesta de soluciones

#	Hallazgo	Problema	Corrección propuesta	Seguridad / Clean Code	Prueba (Fase 2)
1	#1	MD5 es rápido y tiene colisiones conocidas; si se filtra la BD, las contraseñas se	Argon2id (o bcrypt o PBKDF2-SHA512) con sal automática y costo configurable en `apps/api/src/hash.ts`,	La librería verifica en tiempo constante, nunca con ==. Si hay usuarios reales, migrar al iniciar sesión y luego eliminar MD5. Un	El hash no es igual al texto original, dos hashes de la misma

		pueden descifrar con fuerza bruta o tablas arcoíris.	con las funciones `hashPassword` y `verifyPassword`.	solo módulo responsable de las contraseñas.	contraseña son distintos y la verificación acepta la contraseña correcta y rechaza la incorrecta.
2	#2	Si comprometen el proceso, el atacante ya tiene privilegios de root dentro del contenedor.	`USER node` en la etapa final del Dockerfile, con `COPY --chown=node:node`.	Mínimo privilegio; complementar con `read_only: true` y `cap_drop: [ALL]` en el compose.	`docker compose exec web id` muestra un uid distinto de 0; Semgrep y SonarQube dejan de reportar #2.
3	#3, #7, #8, #9, #10, #12	Faltan cabeceras que el navegador usa como segunda línea de defensa (XSS, clickjacking, MIME sniffing, aislamiento) y se revela la tecnología del servidor.	API: `app.disable('x-powered-by')` y helmet con CSP, X-Frame-Options, nosniff, COOP, CORP y Permissions-Policy. Next.js: `poweredByHeader: false` y las mismas cabeceras en `headers()`.	Empezar con CSP en modo Report-Only y pasar a nonces en vez de `unsafe-inline`. Las cabeceras se definen una sola vez, en una constante con nombre y en un único punto de entrada.	`curl -sI` muestra las cabeceras y no muestra X-Powered-By; test con supertest para la API; ZAP baseline ya no reporta esas alertas.
4	#4, #5, #6	pnpm sin políticas de cadena de suministro: puede instalar paquetes recién publicados, de orígenes no confiables o con menos garantías.	En `pnpm-workspace.yaml`: `trustPolicy: no-downgrade`, `blockExoticSubdeps: true` y `minimumReleaseAge: 10080`.	Reduce el riesgo de paquetes publicados con credenciales robadas. Excepciones explícitas con `minimumReleaseAgeExclude`. La configuración vive en el repositorio.	`pnpm install --frozen-lockfile` sigue funcionando y Semgrep deja de reportar #4–#6.
5	#13–#17	Dependencias con CVE conocidas, directas y transitivas.	proxy-addr ≥ 2.0.8 con `trust proxy` 1; next ≥ 14.2.31 o ≥ 15.4.5; jsonwebtoken ≥ 9.0.0 con `algorithms: ["HS256"]`; overrides de lodash ≥ 4.18.0 y qs ≥ 6.16.0.	Un commit por paquete, pruebas después de cada actualización y `pnpm audit --prod` en el CI. Validar la entrada si se usa `_.template` o `qs.stringify` con datos del usuario.	`pnpm audit --prod` deja de reportar #13–#17.
—	#11, #18–#22	Aceptados.	Sin corrección por ahora.	—	—

6. Correcciones (Fase 2)

#	Hallazgo	Corrección aplicada	Prueba que lo demuestra	Commit
1	MD5 para hashear contraseñas (#1)	PBKDF2-SHA512 con 100,000 iteraciones y salt aleatorio de 16 bytes	test/crypto-hash.test.mjs	50a5528
2	Contenedor corre como root (#2)	Directiva `USER node` en los Dockerfiles de api y web	test/docker-security.test.mjs	e1bcac0
3	X-Powered-By expone el framework (#3)	`app.disable('x-powered-by')` en Express y `poweredByHeader: false` en Next	test/security-headers.test.mjs	5823096
4	pnpm sin trustPolicy, blockExoticSubdeps ni minimumReleaseAge (#4, #5, #6)	Políticas de cadena de suministro en pnpm-workspace.yaml y .npmrc	test/dependencies-security.test.mjs	de9a34b
5	CSP ausente o permisiva, anti-clickjacking, nosniff y Permissions-Policy (#7, #8, #9, #12)	API: helmet con una política única (apps/api/src/security-headers.ts). Web: CSP con nonce por petición en apps/web/middleware.ts, sin unsafe-inline ni unsafe-eval en producción, con form-action 'self' y sin el comodín https: en img-src. Cabeceras estáticas en apps/web/csp.mjs	test/security-headers.test.mjs y test/csp-middleware.test.mjs: peticiones HTTP reales a la API y CSP del middleware de Next	5823096, 6bbaad0
6	COOP y CORP ausentes (#10)	`Cross-Origin-Opener-Policy` y `Cross-Origin-Resource-Policy: same-origin`	test/security-headers.test.mjs	9d5bdf0
7	proxy-addr, next, jsonwebtoken, lodash y qs vulnerables (#13–#17)	pnpm-lock.yaml regenerado y commiteado: proxy-addr 2.0.8, next 15.5.28 (React 19), jsonwebtoken 9.0.3, lodash 4.18.1 y qs 6.16.1, con overrides ^2.0.8, ^15.5.24, ^4.18.0 y ^6.16.0. trust proxy 1, JWT limitado a HS256 y Dockerfiles con pnpm install --frozen-lockfile	test/dependencies-security.test.mjs: ejecuta el PoC de cada advisory contra la librería instalada	0629f4d, 9d5bdf0, f67ab60, 50df508
8	SQL Injection (CWE-89)	Consultas preparadas con parámetros	test/sql-injection.test.mjs: UNION SELECT que inyecta un admin, ' OR '1'='1 y DROP TABLE	bf86e64
9	CORS permisivo y stack traces en errores (CWE-942, CWE-209)	Lista de orígenes permitidos y errores genéricos	test/cors-errors.test.mjs: petición desde un origen malicioso y error provocado	8cfc731

10	JWT sin verificar, sin control de roles y cookie sin HttpOnly (CWE-347, CWE-862, CWE-1004)	Verificación criptográfica del JWT, RBAC en las rutas y cookie HttpOnly	test/access-control.test.mjs: tokens con alg none, firmados con un secreto adivinado, con el payload manipulado y con HS512; un viewer intenta escalar privilegios	e5c8303
11	/api/users devolvía el hash de las contraseñas (CWE-256)	Se quitó password_hash de las respuestas de GET /api/users y GET /api/users/:id	test/access-control.test.mjs	78cfabe
12	sonar-project.properties no estaba en el repositorio	Se agregó con projectVersion=fase2, exclusiones, cobertura y SARIF (sin token)	Análisis fase2 con git (revisión 713aaf2)	713aaf2

Cómo se comprobó que las pruebas reproducen el ataque

Cada prueba envía el ataque real: peticiones HTTP a la API con supertest o el PoC del advisory contra la librería instalada. Para demostrar que detectan la vulnerabilidad, se reintrodujo cada una en el código y se volvió a correr la prueba correspondiente:

Vulnerabilidad reintroducida	Pruebas que fallan
jwt.decode en lugar de jwt.verify	5
jwt.verify sin algorithms: ['HS256']	2
passwordHash en la respuesta de /api/users	1
Sin requireRole('admin') en /api/users	4
Cookie de sesión sin HttpOnly	1
SQL concatenado en el login	3
SQL concatenado en la búsqueda de tickets	2
CORS que refleja cualquier origen	3
Stack trace en la respuesta de error	1
API sin helmet	5
CSP con 'unsafe-inline' y 'unsafe-eval'	3
CSP sin form-action	2
lodash 4.17.11, qs 6.15.3 y proxy-addr 2.0.7 (versiones originales)	5
Middleware de Next con 'unsafe-inline' en lugar del nonce	2

Con el código corregido (commit 713aaf2): 67 pruebas, 67 pasan y 0 fallan (pnpm test). Con las versiones vulnerables de las librerías, los 5 PoC de dependencias se ejecutan y sus pruebas fallan.

Pruebas que reproducen los ataques (commit 713aaf2)

```
PS C:\secure-crm> pnpm test
ok 1 - CWE-347 / #15: tokens JWT falsificados son rechazados
ok 2 - CWE-862: un viewer no puede ejecutar acciones de admin/agent
ok 3 - CWE-256: ni siquiera un admin recibe hashes de contraseñas
ok 4 - CWE-1004: la cookie de sesion no es legible desde JavaScript
ok 5 - Refactorizacion Clean Code y Calidad SonarQube (S3358, S4624, S7772, S7744)
ok 6 - CWE-942: un sitio malicioso no puede leer datos de la API con las cookies de la victima
ok 7 - CWE-209: un error provocado no revela stack trace ni rutas internas
ok 8 - Seguridad Criptografica de Contraseñas (CWE-327 / CWE-759)
ok 9 - CSP con nonce por peticion en el middleware de Next (#7)
ok 10 - #16 lodash: inyeccion de codigo via claves de _.template imports (GHSA-r5fr-rjxr-66jc)
ok 11 - #16 lodash: prototype pollution con _.defaultsDeep (CVE-2019-10744)
ok 12 - #17 qs: DoS con isBuffer controlado por el atacante (GHSA-4mjr-xmp4-gh2g)
ok 13 - #17 qs: bypass de arrayLimit con a[]=1,2,3,4 (GHSA-x5fp-wj9c-mxmx)
ok 14 - #13 proxy-addr: un cliente no puede falsificar su IP con X-Forwarded-For (GHSA-jqcg-44mw-7w3h)
ok 15 - #14 next: la version instalada y la del lockfile son >= 15.5.24
ok 16 - #13-#17: ninguna version vulnerable queda en pnpm-lock.yaml
ok 17 - #4-#6: politicas de cadena de suministro de pnpm
ok 18 - Seguridad en Contenedores Docker (CWE-250 / SonarQube docker:S6471)
ok 19 - API: las respuestas HTTP reales llevan las cabeceras de seguridad (#3, #7-#10, #12)
ok 20 - Web: la CSP de produccion bloquea scripts inyectados (#7 y alertas medias de ZAP Fase 2)
ok 21 - Web: next.config aplica las cabeceras estaticas a todas las rutas (#3, #8-#10, #12)
ok 22 - CWE-89: bypass de login con inyeccion SQL
ok 23 - CWE-89: inyeccion en la busqueda de tickets
# tests 67
# pass 67
# fail 0
```

6.2 Verificación DAST: comparación alerta por alerta

Se repitió el mismo escaneo baseline de ZAP de la Fase 1 sobre la versión corregida (docker compose up, http://host.docker.internal:3000) y además un full scan con escaneo activo. Comparación alerta por alerta:

Alerta de ZAP	Riesgo	Fase 1 (8 oct)	Fase 2 (16 oct)
Content Security Policy (CSP) Header Not Set	Media	4 instancias	Corregida
Missing Anti-clickjacking Header	Media	2 instancias	Corregida
Server Leaks Information via X-Powered-By	Baja	4 instancias	Corregida
X-Content-Type-Options Header Missing	Baja	Sistémica	Corregida
Cross-Origin-Opener-Policy Missing	Baja	1 instancia	Corregida
Cross-Origin-Resource-Policy Missing	Baja	Sistémica	Corregida
Permissions Policy Header Not Set	Baja	Sistémica	Corregida
Cross-Origin-Embedder-Policy Missing	Baja	1 instancia	1 instancia (aceptada, #11)
Total altas / medias / bajas		0 / 2 / 6	0 / 0 / 1

Full scan final: 141 reglas pasadas, incluidas las activas (inyección SQL, XSS, path traversal y otras), con 0 altas y 0 medias. La única alerta baja es COEP, aceptada en el hallazgo #11.

Resumen de los reportes de ZAP (antes y después):

ZAP baseline - Fase 1 (8 oct)

Generated on Thu, 8 Oct 2026 16:41:04 · http://host.docker.internal:3000

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	6
Informational	3
False Positives:	0

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	4
Missing Anti-clickjacking Header	Medium	2
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	1
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	1
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	Systemic
Permissions Policy Header Not Set	Low	Systemic
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	4
X-Content-Type-Options Header Missing	Low	Systemic
Modern Web Application	Informational	4
Non-Storable Content	Informational	2
Storable and Cacheable Content	Informational	Systemic

Fase 1: 2 medias y 6 bajas.

ZAP baseline - Fase 2 final (16 oct, commit 713aaf2)

Generated on Fri, 16 Oct 2026 20:07:22 · http://host.docker.internal:3000

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	1
Informational	3
False Positives:	0

Alerts

Name	Risk Level	Number of Instances
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	1
Modern Web Application	Informational	4
Non-Storable Content	Informational	3
Storable and Cacheable Content	Informational	Systemic

Fase 2, baseline: 0 medias.

ZAP full scan - Fase 2 final (16 oct, commit 713aaf2)

Generated on Fri, 16 Oct 2026 20:16:09 · http://host.docker.internal:3000

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	1
Informational	4
False Positives:	0

Alerts

Name	Risk Level	Number of Instances
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	2
Modern Web Application	Informational	5
Non-Storable Content	Informational	4
Storable and Cacheable Content	Informational	Systemic
User Agent Fuzzer	Informational	Systemic

Fase 2, full scan: 0 medias.

Cabeceras HTTP reales de la versión corregida:

Cabeceras HTTP reales (docker compose up)

```
PS C:\secure-crm> curl -sI http://localhost:3000/login
HTTP/1.1 200 OK
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
content-security-policy: default-src 'self'; script-src 'self' 'nonce-MTY0MzEzMjktMDJiMC00YzUwLWFhNTktMWM4Y2YwMGZjODlm' 'strict-dynamic'; style-src
'self' 'nonce-MTY0MzEzMjktMDJiMC00YzUwLWFhNTktMWM4Y2YwMGZjODlm'; img-src 'self' data: blob:; font-src 'self'; connect-src 'self'
http://localhost:4000; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'

PS C:\secure-crm> curl -sI http://localhost:4000/api/health
HTTP/1.1 200 OK
Content-Security-Policy: default-src 'none';base-uri 'none';form-action 'none';frame-ancestors 'none'
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Referrer-Policy: strict-origin-when-cross-origin
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Permissions-Policy: camera=(), microphone=(), geolocation=()
```

7. Preguntas

1. ¿SonarQube encontró algún secreto en tu repositorio? ¿Cómo llegó ahí y qué harías en un proyecto real?

Respuesta: SonarQube no reportó ningún secreto: los 24 issues de la línea base fase1 (3 High, 19 Medium, 2 Low) no incluyen reglas de secretos y no hay Security Hotspots. Sin embargo, sí hay uno en el repositorio: el secreto de los JWT tiene un valor débil por defecto (no se reproduce aquí) escrito en apps/api/src/auth.ts (valor por defecto si falta la variable JWT_SECRET), en docker-compose.yml y en .env.example. Llegó ahí porque el proyecto es un laboratorio y el valor se puso para que la app arrancara sin configuración. Quien lo conozca puede firmar un token de admin válido.

En un proyecto real: un secreto en el repositorio se trata como comprometido aunque el repo sea privado. (1) Revocarlo y rotarlo de inmediato. (2) Eliminarlo del historial de Git (`git filter-repo` o BFG) y limpiar los clones. (3) Moverlo a variables de entorno o a un gestor de secretos, con `.env` en `.gitignore` y un

`.env.example` sin valores. (4) Prevenir que se repita con escaneo de secretos en pre-commit y en CI (por ejemplo gitleaks) y con push protection en el repositorio.

2. ¿Por qué el Quality Gate "Sonar way" habría dado "Passed" en tu primer análisis?

Respuesta: porque «Sonar way» evalúa solo el **código nuevo** (enfoque «Clean as You Code»): sin issues nuevos, hotspots nuevos revisados, cobertura del código nuevo ≥ 80 % y duplicación del código nuevo ≤ 3 %. En el primer análisis no hay una línea base contra la cual medir qué es «nuevo», así que esas condiciones no tienen nada que fallar y el gate da «Passed» aunque el proyecto ya trajera MD5 como hash de contraseñas (Critical, `hash.ts:7`), un contenedor como root y 15 CVEs de dependencias. Por eso la evaluación usa el Quality Gate del curso.

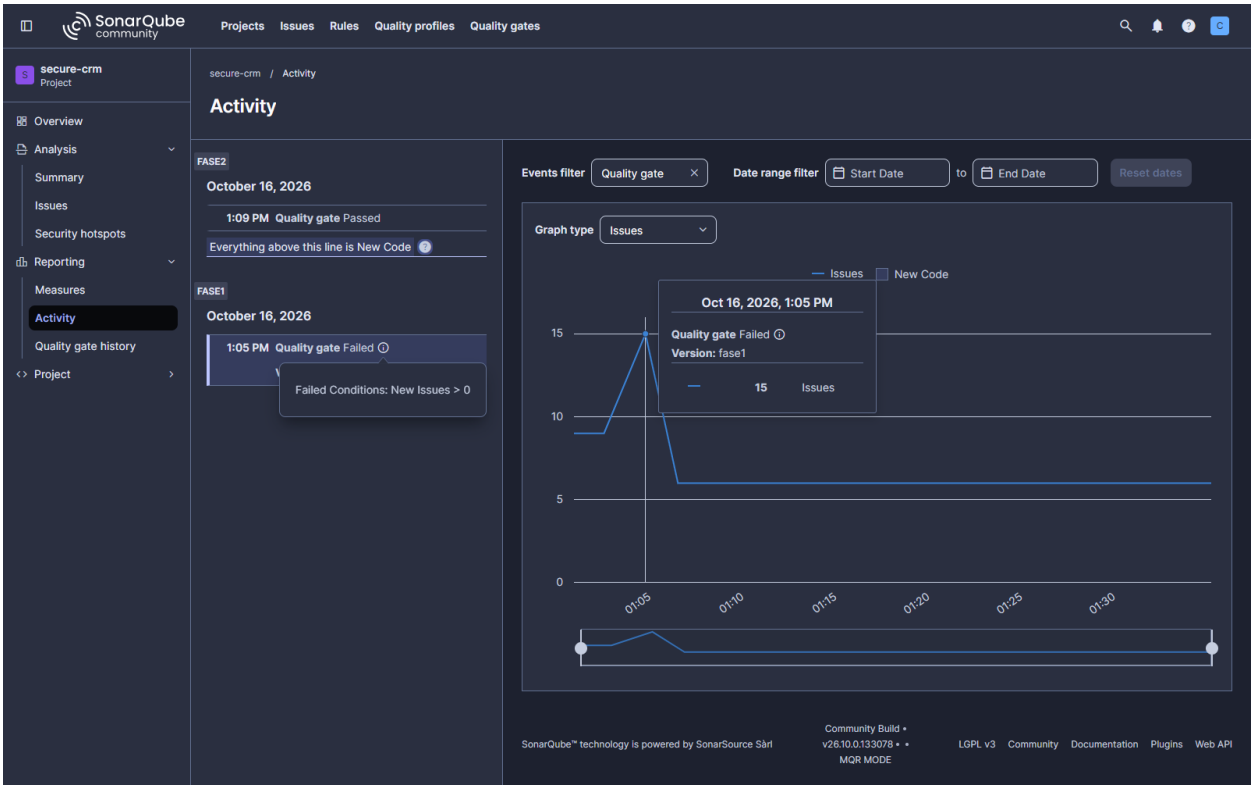
Resultado del gate «Curso Desarrollo Seguro» en fase1: Failed. El gate solo evalúa condiciones sobre código nuevo (Issues > 0, Security Hotspots Reviewed < 100 %, Coverage < 80 % y Duplicated Lines > 3 %). El historial de análisis lo muestra:

• Primer análisis (código original, commit 848d4be): Passed, con Security D. No había línea base, así que nada contaba como nuevo: el mismo efecto que con «Sonar way».

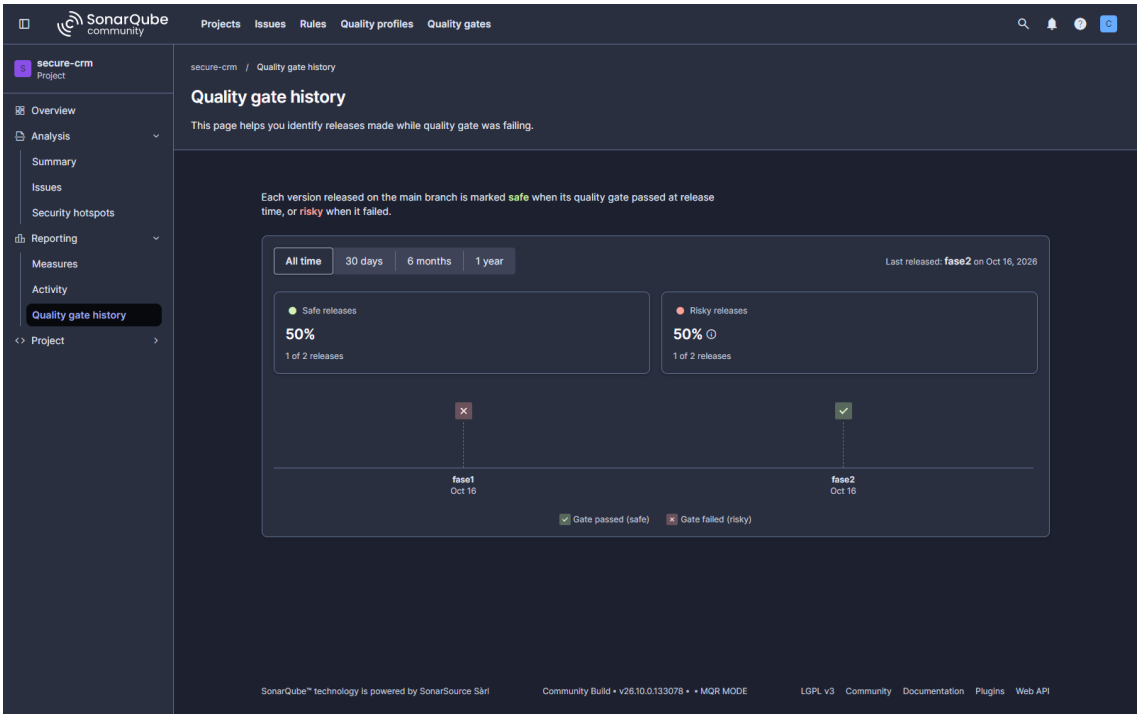
• Análisis «fase1» con los 6 hallazgos de Semgrep importados por SARIF: Failed.

• SonarQube registra el motivo exacto: «Failed Conditions: New Issues > 0».

Condición que falló: New Issues > 0. Los issues importados de Semgrep entraron como nuevos. Coverage y Duplicated Lines no se evaluaron porque ese análisis no agregó líneas de código, y Security Hotspots quedó en 0. Las capturas son del reanálisis del 16 de octubre, que repitió la secuencia original.



Activity del proyecto: fase1 Failed por «New Issues > 0»; fase2 Passed.



Quality gate history: fase1 en rojo y fase2 en verde.

3. ¿Qué vulnerabilidad priorizaste primero y por qué? (severidad vs. alcanzabilidad)

Respuesta: prioricé el **MD5 como hash de contraseñas** (#1): es código propio, está en el flujo de registro/inicio de sesión y, si se filtra la base de datos, todas las contraseñas quedan expuestas; su CVSS es 5.9 solo porque exige antes la fuga de los hashes. En segundo lugar, las dependencias **alcanzables en ejecución**: `proxy-addr` (9.1, vía Express), `next` y `jsonwebtoken`. Dejé para después `shell-quote` (8.1) y `braces` (7.5): aunque su CVSS es mayor que el de `jsonwebtoken` (5.0), solo se usan en build y un atacante no puede alimentarlas. Es decir, el puntaje CVSS mide la gravedad de la falla en abstracto; la alcanzabilidad mide si en nuestra aplicación hay un camino para explotarla.

4. ¿Un 70 % de cobertura significa que el código es seguro? Explica con tus datos.

Respuesta: No. Nuestra línea base tiene 0.0 % de cobertura (350 líneas por cubrir) y, aun así, los 24 issues de seguridad (MD5, Dockerfile sin USER, configuración de pnpm, X-Powered-By, dependencias vulnerables) no dependen de la cobertura. Si subiéramos a 70 % con pruebas que no validan seguridad, esos 24 issues seguirían igual. La cobertura solo dice qué líneas ejecutan las pruebas, no si el código es seguro. Los datos de Fase 2 lo confirman: la cobertura total es 58.9 % (829 líneas) porque las páginas de React no tienen pruebas, y aun así el código nuevo tiene 90.5 %. Lo que demuestra la seguridad no es el porcentaje, sino que cada prueba ejecuta un ataque real y falla si la vulnerabilidad vuelve.